RAJALAKSHMI ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY, Chennai

Department of Cyber Security

2026-2027- Odd Semester

III Year /V Sem

CS23533-FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

SENTINEL AI – Enterprise AI Gateway

Review 1 – 08.09.2026

Jeffrey Antony J (241901041) | Joe Benedict A (241901042)
Supervisor: Dr. S. Bhuvana, Assistant Professor [SG]

Introduction

SentinelAI is a centralized Enterprise AI Gateway for applications using Large Language Models. It operates between enterprise applications and LLM providers such as OpenAI, Gemini and Claude.

The system focuses on governance, monitoring, secure access and intelligent reuse of verified responses.

Enterprise Memory Intelligence can reduce repeated API calls, token usage, response time and operating cost.

Problem Statement

Enterprise applications often communicate directly with LLM providers for every user request.
Repeated prompts can generate redundant API calls, higher token consumption and increased operational cost.
Organizations lack a single layer for centralized authentication, governance, usage monitoring and audit logging.
A managed gateway is needed to control AI access and reuse trusted responses across applications.

Motivation

Selected to address practical challenges that appear as organizations adopt LLM-based applications at scale.
The project combines AI integration with enterprise controls such as authentication, policy enforcement and auditing.
Expected benefits include fewer redundant LLM requests, lower API cost, faster responses and more consistent outputs.
The approach provides a practical foundation for centralized AI management across multiple enterprise applications.

Objectives

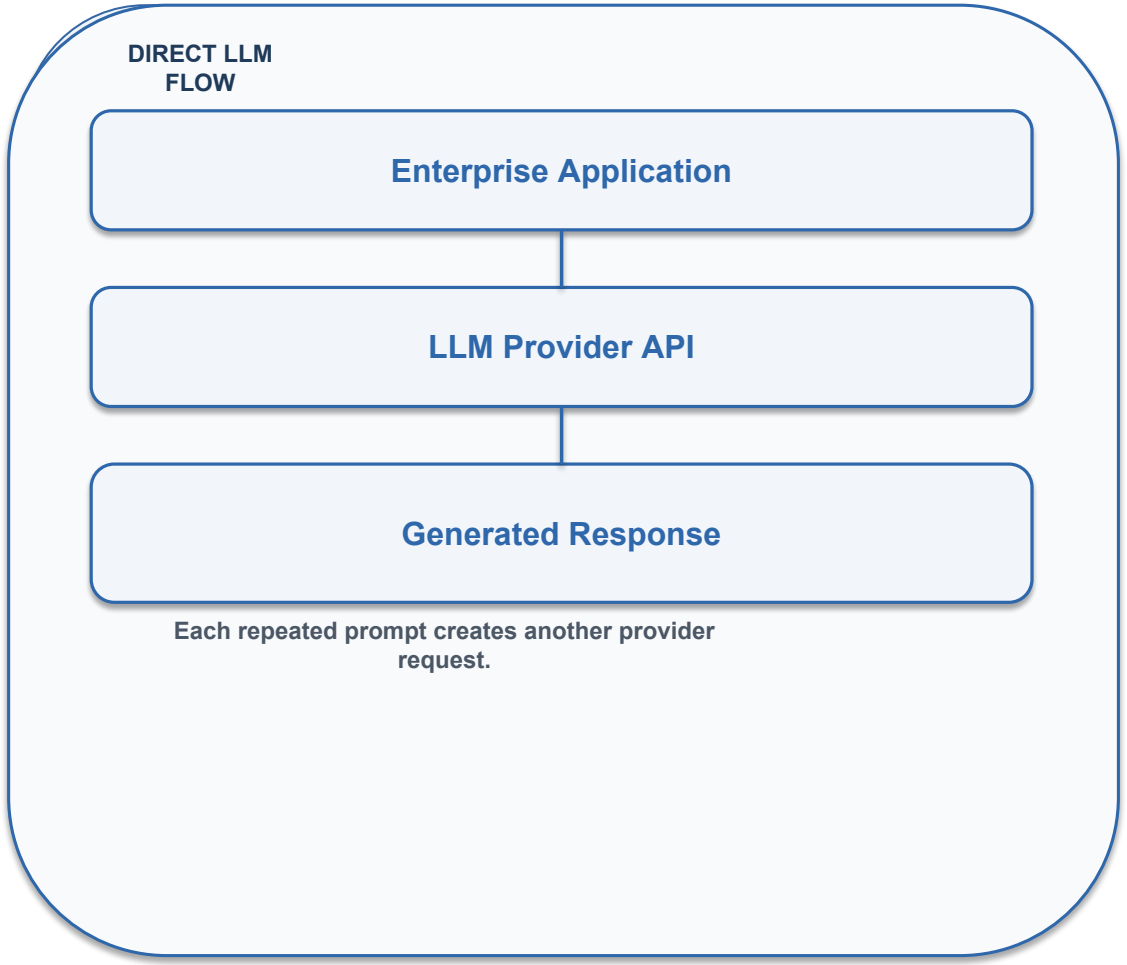
To develop a centralized AI Gateway for enterprise applications.
To reduce unnecessary LLM API calls using Enterprise Memory Intelligence.
To improve response consistency by reusing verified AI responses.
To reduce API costs and token consumption.
To provide organization-based management, analytics and audit logging.

Literature Survey

S.No.	Author & Year	Method / Technique	Key Findings	Limitations
1	Lewis et al. (2020)	Retrieval-Augmented Generation (RAG)	Retrieves relevant external knowledge to improve LLM responses.	Focuses on retrieval; does not provide enterprise-wide response governance.
2	Simon Willison (2023)	Prompt Injection Analysis	Shows how untrusted instructions can manipulate LLM-based systems.	Primarily addresses attack patterns, not centralized enterprise AI management.
3	Harrison Chase (2023)	LLM Orchestration with LangChain	Supports structured development and integration of LLM applications.	Orchestration alone does not provide the proposed memory, governance and cost layer.

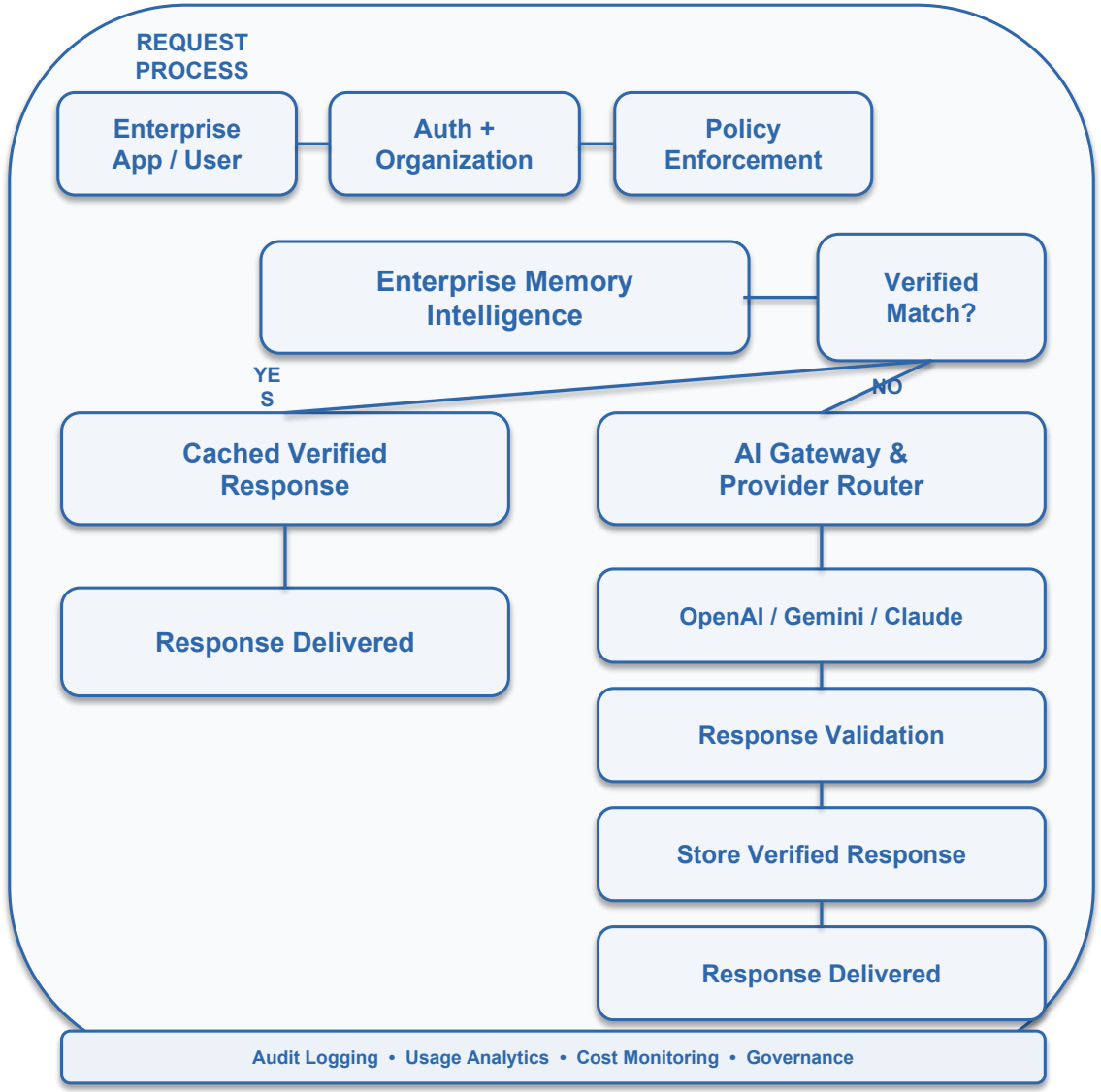
Existing System

Applications directly communicate with LLM providers for each prompt.
Typical flow: application → provider API → generated response.
Advantages: simple integration and direct access to LLM capabilities.
Limitations: repeated API calls, higher token cost, inconsistent control, no centralized audit trail and limited cross-application response reuse.

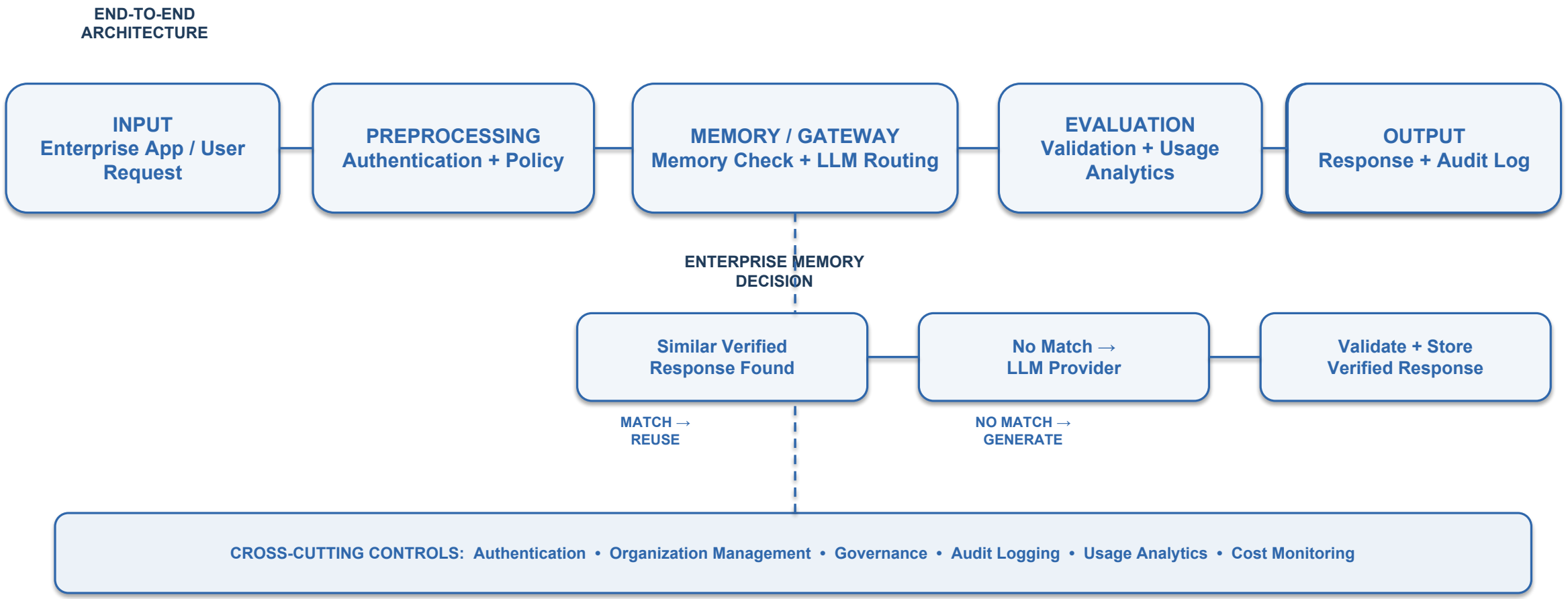


Proposed System

SentinelAI places a centralized gateway between enterprise applications and LLM providers.
Enterprise Memory Intelligence checks for a similar verified response before contacting an LLM.
If no match exists, the request is routed to the selected provider, validated, stored and made reusable.
The platform adds authentication, organization management, audit logging, usage analytics and cost monitoring.



System Architecture / Block Diagram



Project-specific architecture for request routing, memory reuse, validation and enterprise governance.

Methodology

1. User Request & Input

2. Authentication / Policy Check

3. Memory Matching & Processing

4. LLM Routing / Response Generation

5. Response Validation & Analytics

6. Store, Reuse & Deliver Response

Technologies Used

Programming Language: Python 3.13+
Framework / Library: Django, Django REST Framework, React.js / Bun
Database: PostgreSQL / SQLite
Tools / IDE: Postman, Git & GitHub
API / Integration: Gemini API and multiple LLM providers
Hardware: Intel Core i5 or above, 8 GB RAM, 256 GB SSD, stable Internet

Module Description

Module 1 – Authentication & Organization Management

Module 2 – Enterprise Memory Intelligence

Module 3 – AI Gateway & Provider Management

Module 4 – Audit Logging & Usage Analytics

Module 5 – Governance, Cost Monitoring & Dashboard

Preliminary / Expected Results

- **Reduce redundant LLM API calls through verified response reuse.**
- Lower token consumption and operational API cost.
- Improve response time for repeated or similar requests.
- Increase consistency by reusing trusted responses.

- **Centralized authentication and organization-based access.**
- AI usage analytics and cost monitoring.
- Audit logs for AI interactions.
- Scalable integration with multiple LLM providers.

Project Schedule

Activity	Month 1	Month 2	Month 3	Month 4
Literature Survey	✓			
Problem Identification	✓			
System Design	✓	✓		
Implementation		✓	✓	
Testing & Evaluation			✓	✓

Current Progress

- ✓ Literature survey completed
- ✓ Problem identification completed
- ✓ Requirement analysis / methodology finalized
- ✓ Proposed architecture and core modules defined
- ✓ Implementation completed
- ✓ Testing completed
- ☐ Final evaluation completed

Future Work

Complete the SentinelAI implementation and integrate the gateway components.
Implement Enterprise Memory Intelligence for verified response matching and reuse.
Conduct experiments for response time, API calls and token/cost reduction.
Evaluate governance, logging, analytics and multi-provider integration.
Prepare the final system, documentation and comparative evaluation.

Conclusion

SentinelAI proposes a centralized gateway for managing enterprise applications that use LLM services.
Its main contribution is Enterprise Memory Intelligence, which enables verified response reuse and reduces redundant requests.
The expected result is a more controlled, consistent and cost-aware way to integrate multiple LLM providers in an enterprise environment.

References

[1] Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS 2020.
[2] Willison, S. (2023). Prompt Injection Attacks Against Large Language Models. Personal Blog / ACL Workshop.
[3] Chase, H. (2023). LangChain: Building LLM Applications. GitHub / LangChain Documentation